jLWaves

Trapped waves in a stratified circular basin with an island

K.H. Brink
April 16, 2026

0. Overview

This document describes a Julia software package (a set of functions contained in a single ".jl" file) for computing trapped wave functions in a stratified ocean with a sloping bottom. The basin is circular and contains a central island. This geometry allows for resolving waves at an island or trapped at the outer edge of an annulus. More context on these waves can be found in Brink (1999). The package (jLWaves) software allows for a complex modal structure and frequency.

1. Introduction

There exists a set of publicly available Matlab mfiles that can be used to calculate coastal-trapped wave properties in a range of conditions and geometries (https://www2.whoi.edu/staff/kbrink/software/matlab-links/). While these are free for anyone to use, they are only viable for people having access to the Matlab system, which involves paying for a license. Thus, I have been advised that the fair thing to do is to translate these codes so that they can be used in a framework which is freely available to anyone. After some searching around, experimentation and advice from colleagues, I have settled on the Julia system (https://julialang.org/learning/). This well-regarded system has many of the same qualities as Matlab, and is relatively easy to learn, knowing Matlab.

There are, of course, some differences from the Matlab system. One is that the core Matlab system already includes some software packages (such as for interpolation, graphics, and searching), a considerable convenience. With Julia, many of these capabilities require the activation of additional free software packages. This is really only a minor inconvenience once one installs the appropriate package. Further, there are a number of other aspects where the two systems vary (https://docs.julialang.org/en/v1/manual/noteworthy-differences/), some of which take some getting used to.

2. Using the Julia-based system:

After installing Julia on your computer, you will need to install a number of software packages in order to use this code. This only needs to be done only once, but the packages need to be activated (with the "using" command) any time you want to use them. For the present wave code (in file jLWaves.jl), one needs the following packages:

CairoMakie

GLMakie
Interpolations
JLD2
Makie
NLopt
Printf
SparseArrays.

Plus the "Revise" package is also handy when writing and revising code. One adds a package simply as follows:

```
julia> using PkG
julia> Pkg.add("CairoMakie")  .
```

In order to use the code in the files, one needs first to establish a "module" which defines the working space that you will use. The module file (ModLW.jl) that works for this island wave code contains simply the following lines:

```
module ModLW

    using Revise
    using Makie
    using CairoMakie
    using JLD2
    using Interpolations
    using NLopt
    using Printf
    using SparseArrays

    # println("You may want to change directories")

    cd("C:\\Projects\\jwaves\\jLWaves")

end
```

(The # marker denotes a comment). This file defines the module/working space, activates the packages that will be needed, and sets the directory to where the desired wave ("L" for "isLand") code is to be found. The user will likely have a different path for this.

A typical session might begin as follows. First a module is established by using the "include" command. This activates the above code, and places you in the desired directory. Note that for this purpose, you have to use double slashes, not single ones. Next, you include the wave code, again with an "include" command, but now, you do not need to state the whole path, because you are in the desired directory. Next, load (or create: see below) the tuple (an array of various arrays) that gives the wave code its needed information (more on this below). For a given wave problem, the code files (like

jLWaves.jl) contain all of the individual functions you need to proceed. Finally, you can run the code by calling the central function jLWavesM. Thus (I have placed in red what Julia provides, in blue what you, as a user type in, and in black, the outputs):

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | |/ _` | |
  | | |_| | | | (_| |  |  Version 1.12.0 (2025-10-07)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

julia> include("C:\\Projects\\jwaves\\jLWaves\\ModLW.jl")
Main.ModLW

julia> include("jLWaves.jl")
inputf (generic function with 1 method)

julia> testi = load_object("testi.jld2");

julia> testi
(100, 30, [5.0e-5 2.0e-6], 1, NaN, 1, NaN, 0.0001, 60.0, 0.0001, 1, 1, 1, 4, [10.0 20.0 30.0 60.0], [10.0
100.0 600.0 600.0], 2, [10.0 60.0], [0.02 0.02], 2, 1000.0, 1.0, [5.0e-5 5.0e-5], 0.0, 20.0, 0.0, 10000.0,
10000.0, 1000.0, 1000.0, 1, 0)

julia> jLWavesM(testi)
nn, mm = 100,  30

Free surface
Open BC at r = rmax

f = 1.000e-04 rad/sec

 Max consistency ratio = 7.386 at r = 0.52 km
        This should be kept less than one, and definitely less than 10


Wavenumber = 1

Iterations:
wr, rrr = 5.0000e-05 2.0000e-06im  1.9061e-33
wr, rrr = 5.1250e-05 2.0000e-06im  1.7733e-33
wr, rrr = 5.1250e-05 2.0500e-06im  1.7732e-33
wr, rrr = 5.2500e-05 2.0500e-06im  1.6205e-33
wr, rrr = 5.3750e-05 2.0750e-06im  1.4505e-33
```

```
wr, rrr = 5.3750e-05 2.1250e-06im  1.4505e-33
wr, rrr = 5.5000e-05 2.1875e-06im  1.2649e-33
wr, rrr = 5.7500e-05 2.2125e-06im  8.4736e-34
wr, rrr = 6.0625e-05 2.2938e-06im  2.4033e-34
wr, rrr = 6.1875e-05 2.4063e-06im  3.1962e-35
wr, rrr = 6.5938e-05 2.5719e-06im  3.6537e-33
wr, rrr = 6.7500e-05 2.5125e-06im  3.1276e-33
wr, rrr = 5.8125e-05 2.2688e-06im  7.3252e-34
wr, rrr = 6.4375e-05 2.4313e-06im  1.3673e-33
wr, rrr = 5.9688e-05 2.3094e-06im  4.2745e-34
wr, rrr = 6.2813e-05 2.3906e-06im  4.5466e-35
wr, rrr = 6.4063e-05 2.5031e-06im  9.1378e-34
wr, rrr = 6.1484e-05 2.3461e-06im  8.5440e-35
wr, rrr = 6.3203e-05 2.4508e-06im  1.6181e-34
wr, rrr = 6.1914e-05 2.3723e-06im  2.9543e-35
wr, rrr = 6.0977e-05 2.3879e-06im  1.7095e-34
wr, rrr = 6.2354e-05 2.3899e-06im  5.7422e-36
wr, rrr = 6.2393e-05 2.3560e-06im  8.2448e-36
wr, rrr = 6.2832e-05 2.3736e-06im  5.0320e-35
wr, rrr = 6.2144e-05 2.3726e-06im  1.2131e-35
wr, rrr = 6.2603e-05 2.3733e-06im  1.7893e-35
wr, rrr = 6.2258e-05 2.3728e-06im  7.7559e-36
wr, rrr = 6.2219e-05 2.4068e-06im  7.0277e-36
wr, rrr = 6.2314e-05 2.4239e-06im  4.0835e-36
wr, rrr = 6.2343e-05 2.4495e-06im  2.8079e-36
wr, rrr = 6.2477e-05 2.4327e-06im  6.2040e-36
wr, rrr = 6.2412e-05 2.4262e-06im  4.5011e-36
wr, rrr = 6.2401e-05 2.4858e-06im  1.9382e-36
wr, rrr = 6.2425e-05 2.5337e-06im  1.2868e-36
wr, rrr = 6.2355e-05 2.5570e-06im  1.3218e-37
wr, rrr = 6.2327e-05 2.6223e-06im  2.9087e-37
wr, rrr = 6.2438e-05 2.6411e-06im  1.6877e-36
wr, rrr = 6.2414e-05 2.5932e-06im  6.2906e-37
wr, rrr = 6.2345e-05 2.6165e-06im  1.4906e-37
wr, rrr = 6.2286e-05 2.5803e-06im  6.5154e-37
wr, rrr = 6.2382e-05 2.5900e-06im  1.4790e-37
wr, rrr = 6.2393e-05 2.5304e-06im  7.4787e-37
wr, rrr = 6.2357e-05 2.5950e-06im  1.6118e-38
wr, rrr = 6.2330e-05 2.5620e-06im  1.5850e-37
wr, rrr = 6.2369e-05 2.5830e-06im  4.9159e-38
wr, rrr = 6.2370e-05 2.6210e-06im  2.4307e-37
wr, rrr = 6.2359e-05 2.5730e-06im  3.5427e-38
wr, rrr = 6.2347e-05 2.5850e-06im  3.7412e-39
wr, rrr = 6.2336e-05 2.5860e-06im  3.9008e-38
wr, rrr = 6.2344e-05 2.6070e-06im  7.4980e-38
wr, rrr = 6.2355e-05 2.5815e-06im  5.6141e-39
```

```
wr, rrr = 6.2346e-05 2.5715e-06im  3.8205e-38
wr, rrr = 6.2354e-05 2.5891e-06im  2.2081e-39
wr, rrr = 6.2345e-05 2.5926e-06im  1.2693e-38
wr, rrr = 6.2353e-05 2.5843e-06im  7.9199e-40
wr, rrr = 6.2360e-05 2.5884e-06im  1.1826e-38
wr, rrr = 6.2350e-05 2.5858e-06im  3.5420e-40

Converged!

wr, rrr = 6.2350e-05 2.5858e-06im  3.5420e-40

Do you want to save this information? 0 (no) or 1 (yes) 0

julia>
```

This example includes a simple running of the basic island wave code (with bottom friction in this case), and shows how, once a wave mode is detected, you have the option of saving the results to a file that you can name. This function also leads to plots of the modal structure, "dispersion curve" (if more than one wavenumber is called for) and other items of interest. You do not see any of the needed "using" commands because they have been incorporated into the modLW.jl file.

In order to create the needed tuple, use the function (for this case) jLWavesSetup (within file jLWaves.jl). You call the function, and it asks you a sequence of questions that will build up the tuple exi:

```
julia> exi = jLWavesSetup();

     Island-Trapped waves with complex frequency

This function will ask you a sequence of questions
    that are used to build the input tuple.

How many total gridpoints do you want in the radial direction? (nn) 100
How many total gridpoints do you want in the vertical? (mm) 30
Enter first guess at real and complex parts of frequency (2-elemnent array) (wg) (1/sec) [3e-5 1e-6]
Enter 0 for a rigid lid, 1 for a free surface (del) 1
Enter 0 for a closed r =rmax boundary, 1 for an open (iobc) 1
Enter the Coriolis parameter (f) (rad/sec) 1e-4
Enter the maximum radius (rmax) (km) 75
Enter the nominal fractional accuracy for the frequency solution (eps) 0.0001
Enter the number of frequencies to be computed (npts) 3
Enter the first azimuthal wavenumber to use (dnw) (integer) 1
```

Enter the wavenumber increment to use after nwz (dnw) (integer) 1

When entering topography, the first radius falls at the coast

How many radius, depth pairs will you provide (ndep >=1) 4
Array of radial distances for depth values (rdep in km) (dimension ndep) [20 30 50 75]
Array of depths corresponding to rdep (depr in m) [10 100 500 500]
Number of radius, bottom friction pairs to read (nR) 1
Radial distances for bottom friction values (rR in km) 20
Array of bottom friction values corresponding to rR (rr in cm/sec) 0.01
Depth increment for input of Nsquared values? (zr in m) 1000
Exponential tail length for Nsquared extrapolation (alph in km) 1
Nsquared values starting at the surface (nsqr in rad^2/sec^2) (nnsq values) [5e-4 5e-4]
Input peak value of mean azimuthal flow (vzero: cm/sec) 20
Input radial distance to peak mean flow (km) 30
Input depth of peak mean flow (m) 0
Downward exponential scale of mean flow? (m) 200
Upward exponential scale of mean flow? (m) 200
Offshore exponential scale of mean flow? (km) 10
Onshore exponential scale of mean flow? (km) 10
Enter 1 for undisturbed Nsquared offshore, 0 for onshore 1
Enter 0 to skip pauses to see graphics, 1 to see graphics during execution 0

julia>

The function then provides a plot of bottom friction, topography and mean azimuthal flow.
Note that this example does not display the completed tuple at the end. If you do not wanted to see it, just remove the semicolon after the () when you call the function. Alternatively, you could just enter the tuple name exi, and it would display the tuple.

```
julia> exi
(100, 30, [3.0e-5 1.0e-6], 1, NaN, 1, NaN, 0.0001, 75.0, 0.0001, 3, 1, 1, 4, [20.0 30.0 50.0 75.0], [10.0 100.0 500.0 500.0], 2, [20.0 75.0], [0.01 0.01], 2, 1000.0, 1.0, [0.0005 0.0005], 20.0, 30.0, 0.0, 200.0, 200.0, 10.0, 10.0, 1, 0)
```

You can save this tuple to the current directory with the command:

```
julia> save_object("sampli.jld2", exi)
```

The above actions, together, create the tuple exi, plot friction, depth and mean flow as a figure, and then save the tuple to a file named sampli.jld2 .

Finally, you can modify an input tuple by using the function "jLWavesFinch":

```
julia> xx = jLWavesFinch(exi)

    Function to create modified input tuple for an island

First you need to select what you want to change.

    Options are:
      g:  Grid size
      w:  Initial Frequency guess
      b:  Boundary conditions
      f:  Coriolis parameter
      x:  Domain size
      d:  Dispersion curve definition
      e:  Nominal accuracy
      h:  Depth profile
      R:  Bottom friction
      n:  Stratification
      v:  Mean flow field
      p:  Pauses during execution

Any arrays are row arrays, not column arrays.

Please type in the appropriate letter and hit return
w

Old frequency guess (real and complex parts) (sec^-1) [3.0e-5 1.0e-6]
 Enter new real and complex frequency parts (2-element array) (sec^-1) [2e-5 1e-6]
(100, 30, [0.002 1.0e-6], 1, NaN, 1, NaN, 0.0001, 75.0, 0.0001, 3, 1, 1, 4, [20.0 30.0 50.0 75.0], [10.0
100.0 500.0 500.0], 2, [20.0 75.0], [0.01 0.01], 2, 1000.0, 1.0, [0.0005 0.0005], 20.0, 30.0, 0.0, 200.0,
200.0, 10.0, 10.0, 1, 0)

julia>
```

Note that the blue/red/black color convention still holds. Note, too, that when you provide depth (or friction coefficient *R*) values, you do not have to give values extending all the way to the offshore boundary. The code will just fill out the rest of the topography with the last value given (1000 m in this case).

More explanations are given in the following sections, but this section was mainly meant to show the basics of using this code in Julia.

3. The Physical Problem

We seek to find the free wave solutions for linearized island-trapped waves at a circular island. The depth $h(r)$ is uniform in the azimuthal (θ) direction. The offshore (radial) direction is r ($r = r_0$ is the coast) and the vertical direction is z. The bottom is at $z = -h(r)$ and the origin is at the island center. Dissipative effects are limited to the infinitesimally thin bottom boundary layer.

The equations of motion in cylindrical coordinates are:

$$v_t + uv_{0r} + v_0 u_\theta / r + uv_0/r + wv_{0z} + fu = -\rho_0^{-1} r^{-1} p_\theta + \rho_0^{-1} \tau^r{}_z \tag{1a}$$

$$u_t + v_0 u_\theta / r - 2v_0 v/r - fv = -\rho_0^{-1} p_r + \rho_0^{-1} \tau^\theta{}_z \tag{1b}$$

$$0 = -p_z - g\rho_2 \tag{1c}$$

$$r^{-1}(ru)_r + r^{-1} v_\theta + w_z = 0 \tag{1d}$$

$$\rho_{2t} + u\,\rho_{1x} + v_0 \rho_{2\theta} / r + w\rho_{1z} = 0 \tag{1e}$$

where (u, v, w) are the perturbation velocity components in the (r, θ, z) directions, respectively, p is perturbation pressure, f is the (constant) Coriolis parameter, $v_0(r, z)$ is a steady azimuthal mean flow, density is broken up as: $\rho_0 + \rho_1(r, z) + \rho_2(r, \theta, z, t)$ and $\rho_0 >> \rho_1 >> \rho_2$. The $2v_0 v / r$ term in (1b) comes from expanding $(v_0 + v)^2$. In all cases, subscripted independent variables represent partial differentiation.

The mean azimuthal flow $v_0(r, z)$ obeys

$$-v_0 (f + v_0 / r) = -\rho_0^{-1} p_{0r} \tag{2a}$$

so that

$$v_{0z} (f + 2v_0 / r) = -g\,\rho_0^{-1}\,\rho_{1r}\ . \tag{2b}$$

Turbulent stresses in the r, θ directions are given by τ^r, τ^θ, and the bottom stress ($\tau_B{}^r$, $\tau_B{}^\theta$) is related to the interior bottom velocity by

$$\tau_B{}^r = \rho_0 R\, u_B, \qquad \tau_B{}^\theta = \rho_0 R\, v_B, \tag{3 a, b}$$

where $R(r)$ is a bottom resistance coefficient having units of velocity, and a subscript "B" denotes a quantity evaluated at the bottom. We also define

$$N^2 = -g\rho_0^{-1}\,\rho_{1z} \qquad \text{and} \tag{4a}$$

$$M^2 = -g\rho_0^{-1}\,\rho_{1r}\,. \tag{4b}$$

The problem is solved by assuming that

$$p = p'(r, \theta)\, exp[i(\omega t + m\theta)], \text{ etc.,} \tag{5}$$

where the frequency ω can be complex and the azimuthal wavenumber m is a real integer. Henceforth, the "primes" will be dropped. Using this form, the problem can be reduced to a single partial differential equation for pressure (analogous to that in Brink, 2006):

$$\begin{aligned} 0 = {} & \omega' p_{rr} + p_{rz} (-2 s\omega'^2) + p_{zz} \omega' (s^2 + G/N^2) \\ & + p_r [\omega' \Gamma + \omega'_r - s\, \omega_z + m f_1 / r - mf^*/r] \\ & + p_z [s\, \omega' \Gamma - (s\omega')_r - s(s\omega')_z + G (\omega' / N^2)_z + m s (- f_1 + \omega'^2 / f_1) / r] \\ & + p [mf_1 \Gamma / r + m(f_1 / r)_r - ms (f_1 / r)_z - \omega' m^2 / r^2] \end{aligned} \tag{6}$$

where

$$\omega' = \omega + mv_0 / r \tag{7a}$$
$$s = M^2 / N^2, \tag{7b}$$
$$f_1 = f + 2v_0 / r\ , \tag{7c}$$
$$f_2 = f + v_{0r} + v_0 / r\ , \tag{7d}$$
$$f^* = f_2 - sM^2 / f_1\ , \tag{7e}$$
$$G = f_1 f^* - \omega'^2\ , \tag{7f}$$
$$\Gamma = (-G_r + s\, G_z) / G - s_z - 1/ r\ , \tag{7g}$$

This equation and the notation are similar to those of Mooers (1975 a, b), except that these are in cylindrical coordinates and apply to hydrostatic conditions.

The mean azimuthal velocity field is taken to be a Gaussian in r and z, centered at $r = r_v$, $z = -z_v$.

$$v_0(r, z) = \langle v \rangle \exp\{-[(r-r_v) / D^r]^2\} \exp\{-[(z - z_v) / D^z]^2\} \tag{8}$$

where $\langle v \rangle$ is the peak value, and the horizontal scale D^r has different values onshore and offshore of r_v, while the vertical scale D^z is allowed to be different upward and downward of z_v. If you desire to have a different initial velocity form, you can simply change the code in function jLWavesVzcal. Velocity gradients and density are computed numerically given v_0, so that no other code need be changed.

As noted by Brink, (1982) or Dale et al. (2001), equation (6) admits a spurious solution at $\omega' = f$ when $v_0 = 0$. The presence of this mode also distorts modal structures and dispersion curves for nearby frequencies. Typically, this spurious pressure mode is vertically uniform and it decays only slowly with distance offshore.

The boundary conditions are as follows:

z = 0:

$$w = \delta\, (g\rho_0)^{-1}[p_t + v_0 p_\theta / r], \tag{9}$$

where $\delta = 1$ for a free surface condition and $\delta = 0$ for a rigid lid condition. In terms of pressure,

$$0 = \omega'(s^2 + G /N^2)\, p_z \;\; - s\, \omega'\, p_x \;\; + [\omega'\delta\, G /g - m\, s\;\, f_1 / r\,]\, p \tag{10}$$

z = - h(r):

$$0 = w + h_r u + \; r^{-1}(r\, U_E)_r + \; r^{-1} V_{E\theta} \;, \tag{11}$$

where the Ekman transport is given as

$$U_E = - RG^{-1}\, (f_1 v + i\omega' u)_B \;, \tag{12a}$$

$$V_E = - RG^{-1}\, (i\omega' v - f^* u)_B \;, \tag{12b}$$

and the subscript "B" denotes a quantity evaluated at the bottom (along $z = -h(r)$). In terms of pressure,

$$\begin{aligned}
0 = {} & p_{rBr}\, (R /G)\, (f_1 f^* + \omega'^2) \;\; + p_{zBr}\, (R /G)(-2s\omega'^2\,) \\
& + p_{zB}\, \{i\, \omega'(G + s\, M^2 - M^2\, h_r)\, /N^2 \;\; - s\omega'(G\, f_1)^{-1}[f_1(\omega' R)_r + \omega'(f_1 R)_r] \\
& \qquad +(R/G)[4G_r s\omega'^2\, /G \;\; - \omega'(s\omega')_r - f_1(s\omega'^2/f_1)_r \\
& \qquad + (s\omega'/r)(-2\omega' + mf^* + m\, \omega'^2/f_1)]\} \\
& + prB\, \{-i\omega'(s-hr) + [-\omega'(\omega' R)r + f^*(f1R)r]\, / G \\
& \qquad +(R/G)[-2G_r\, (f_1 f^* + \omega'^2)/G + \omega'\omega'_r + f_1 f^*_r \;\; + (f_1 f^* + \omega'^2 - 2m\omega' f^*)/r]\} \\
& + p_{Br}\, (R/G)\, (2m\omega' f_1\, / r) \\
& + p_B\, \{\, -imf_1(s-h_r)/r + m(rG)^{-1}[f_1(\omega' R)_r + \omega'(f_1 R)_r] \\
& \qquad +(R/G)\, \{\, -G_r(4mf_1\omega')\, /(rG) + m\, (\omega' f_{1r} + f_1\omega'_r)\, / r \\
& \qquad\quad -m^2(f_1 f^* + \omega'^2)\, / r^2]\}
\end{aligned} \tag{13}$$

r = r₀:

A closed boundary condition gives no net flux across the coastal boundary:

$$0 = \;\; U_E + \int_{-h}^{0} u\, dz \approx \;\; U_E + uh \tag{14}$$

When there is bottom friction ($U_E \neq 0$), this condition really only makes sense if the vertical variations in u are negligible over the depth $h_0 = h(r_0)$. With no bottom friction, this boundary condition is valid regardless of the coastal wall height.

Stated in terms of pressure, this condition is:

$$0 = - i\,\omega' h_0\, p_r + (i\, s\omega')h_0\, p_z - (\, i\, m\, f_1 h_0 /r)\, p \qquad (15)$$

$$+RG\, G_B^{-2}\, \{ - (f_1 f^* + \omega'^2)\, p_{rB} + (2\, s\omega'^2)\, p_{zB} - (2m\omega' f_1 / r)\, p_B \}$$

r = L:

The offshore boundary condition is an adaptation of the approximate open condition introduced by Brink (1982):

$$0 = (ru)_r \qquad . \qquad (16)$$

This condition seems to work well as long as the outer grid boundary ($r = L$) occurs far offshore of where the modal solutions have substantial structure. The condition, as used, requires that the bottom is flat at the offshore boundary. The condition in terms of pressure is:

$$0 = - \omega'\, p_{rr} + s\omega'\, p_{zr} + p_r\, (\omega' G_r / G - \omega' / r - \omega'_r - m f_1 / r) \qquad (17)$$

$$+ p_z\, [-s\omega' G_r / G + (s\omega')_r + s\omega' / r]$$

$$+ p\, [m f_1\, G_r / (rG) - m (f_1 / r)_r - m\, f_1 / r^2]$$

At the outer boundary, one can also apply a closed boundary condition. If there is no bottom friction, this condition is simply

$$u = 0 \qquad . \qquad (17a)$$

This boundary condition is then valid regardless of the water depth at $r = L$. If there is bottom friction $(R \neq 0)$, then the condition is

$$0 = \int_{-h}^{0} u dz + U_E \cong uh + U_E \qquad (17b)$$

where the approximation is only valid if the water is shallow enough that u does not vary substantially over depth h. Thus, it is important that if you have a closed outer boundary condition (as in a lake), the coastal wall is not very high (certainly less than about 30 m). Expressed in terms of pressure, the closed outer boundary condition becomes

$$0 = p_r\, (-i\omega' h / G) + p_z\, [i\omega' h\, M^2 / (N^2 G)] + p[-im\, f_1 h / (rG)] \qquad (17c)$$

$$+(R/G_B^2)\{\, p_{rB}\, (2i\omega' f^*)_B + p_{zB}\, [\omega' M^2 (\omega'^2 / f_1 - f^*) / N^2]_B + p_B\, [im\, (f_1 f^* + \omega'^2] / r]_B \}$$

The problem is now expressed in stretched coordinates that map the problem into a rectangular domain (e.g. Wang and Mooers 1976). Specifically,

$$\varphi = z / h(r) \tag{18a}$$

and

$$r' = r. \tag{18b}$$

The problem is then solved numerically via resonance iteration. An arbitrary forcing is applied through the surface boundary condition (essentially, a localized wind stress curl near the grid center), and ω is searched for a maximum response so as to establish a complex frequency corresponding to a given wavenumber m. This search makes use of the function "NLopt.optimize" from the NLopt package.

4. Using the software:

Examples of what your Julia screen will look like as you run the different functions are shown in section 2. The main routine, "jLWavesM", requires an input tuple that can be generated by "jLWavesSetup" and modified by "jLWavesFinch". When you run "jLWavesSetup" or "jLWavesFinch", you simply need to answer the questions posed. Do not forget to use square brackets when entering more than one number on a line.

When running "jLWavesM", you will first be shown a few simple parameter values (see the example in section 2). You also get a section plot showing topography, stratification (minus a constant background value), and the mean flow. If f^*/f is negative anywhere, f^* will also be plotted (in red) on the density plot. If you are allowing pauses during the run, you are advised of the 10-second pauses as they occur. During iteration toward a solution, the azimuthal wavenumber m is displayed, and the program displays complex frequency and the inverse resonance parameter (rrr = inverse of the integral of $r|p|^2$ over r and z) for each ω iteration. When you are at a resonance (free mode), rrr becomes very small. You can thus watch the convergence onto a complex free modal frequency. Convergence to your given tolerance is then announced, and the free mode pressure is the plotted as a section. A pause can also be announced at this point. You will then also get section plots of u, v, w, ρ, energy diagnostics in arbitrary units (energy pool sizes and transfers: the dissipation integral should be a negative quantity), and line plots of p, v, u at the surface. If you set the program to run more than one frequency/wavenumber, it will then display the next wavenumber and go on from there.

Before any plots are made, the pressure field is normalized (Brink, 1999) so that

$$1 = \int_{-h(r_0)}^{0} |p(r_0, z)|^2 dz + \int_{r_0}^{L} h_r |p(r, -h)|^2 dr - \eta \int_{-h}^{0} |p(L, z)|^2 dz \,| \tag{19}$$

where $\eta = 1$ with a closed offshore boundary and $\eta = 0$ for an open boundary. This means that, in the following plots, pressure has units of ($cm^{-1/2}$), velocity has units of ($cm^{-3/2}$ sec gm^{-1}), and density has units of ($cm^{-5/2}$ sec^2). Relations in Brink (1999) can be used to estimate wind coupling coefficients when $R = 0$ and $\langle v \rangle = 0$, given his normalized modal form.

If you have set the program to compute more than one frequency/wavenumber pair, the program will also draw a complex dispersion curve once all pairs have been obtained. This is not a true curve, because solutions only occur at integral values of m. However, solutions, hopefully in the same family (e.g., as defined perhaps by number of zero crossings), are connected by dashed lines .You will be asked at the end of the run whether you want to save the results as a jld2 file.

5. Some Notes on using the software:

The $r = r_0$ boundary condition is closed. This causes no issues when $R = 0$, since it just imposes $u = 0$ at each depth, a reasonable condition at a coastal wall. When $R \neq 0$, bottom friction is applied, and the bottom Ekman transport that reaches the coastal wall is balanced by an evenly distributed interior flow. Thus, there is an assumption that the water is shallow enough that $|u_z / u|$ is small. This would be expected if $(f/N)\, L^r$ is large relative to h_0, where L^r is a representative modal scale in the r direction. This means that, with bottom friction, the boundary water depth should be probably substantially less than 50 m. Further, see Mitchum and Clarke (1986) for aspects of boundary layer theory that affect one's choice of wall height. If there is no bottom friction, then the coastal wall can have any depth.

At the offshore boundary, the bottom should be flat, but the water does not need to be shallow when an open boundary condition is used. The bottom need not be flat there with a closed boundary condition, but the water needs to be shallow there when there is bottom friction. For island-trapped modes, care should be taken to make sure that the offshore boundary is located far from the main modal structure's location of maximum amplitude.

The numerical grid extends one point outside the r, z domain at each end (nn is the number of r grid points, and mm the number of z (or, equivalently φ) grid points. There is also a grid point beyond each boundary. Thus, for example, nn = 5 gives one interior grid point, 2 on boundaries and 2 outside the domain.

In choosing a grid size, caution is advised. One should experiment with grid sizes in order to be confident of a good answer. I find that nn = 80-120, mm = 20-30 seems to work well in many coastal and island cases. The number of operations or amount of memory needed goes something like nn*mm*mm. There is a test for hydrostatic consistency (the ratio computed is $h_r \Delta r/(h \Delta \varphi)$), and results seem best if the ratio given is less than around 1-5. This ratio gives a measure of when you might be in trouble estimating the horizontal pressure gradient in sigma coordinates.

The mean azimuthal flow is assumed to be Gaussian (with different onshore, offshore, up and down length scales, as well as a variable radius and depth for the position with maximum velocity). This provides a fair bit of flexibility. Once the velocity field is computed, the density structure is computed

using a variant of the thermal wind equation (2b). The density field can be left undisturbed at either the offshore or onshore end of the domain. The program will alert you (by blue "+" signs) to any regions where your choices have led to gravitationally unstable vertical density gradients, i.e., when $(\rho_0 + \rho_1)_z > 0$.

There is substantial documentation within the function "jLWavesM" for further detail.

Results for pressure, velocity and density are plotted. The smaller of the real or imaginary is suppressed if it is too small (maximum absolute value less than about 1000 times less than the other's maximum absolute value, as it is set now). This ratio is controlled by the value of "ratnplot" in "jLWavesUvwr" and "tol" in "jLWavesM".

The search can be very inefficient if you give it 0 as the starting guess for real or imaginary frequency. Better to give it a very small nonzero value. For example, if you are seeking a mode with a strictly real frequency (say, when R and v_0 are both zero), then a good starting guess might be $1\times10^{-5} + i\ 1\times10^{-13}$ sec^{-1} rather than simply $1\times10^{-5} + i\ 0\ \text{sec}^{-1}$. Similarly, if you are seeking a damped wave frequency (e.g., if $R \neq 0$ and $v_0 = 0$), then it would be well to start with a guess like $(1 + i)\times2\times10^{-5}\text{sec}^{-1}$, rather than $(2\times10^{-5} + i\ 1\times10^{-10})\ \text{sec}^{-1}$, for example (i.e., use similar real and imaginary magnitudes, rather than a large and very small value). Similarly, if you are seeking an unstable mode, you might want to start searching in a similar way, e.g., $(1 - i)\times5\times10^{-6}\ \text{sec}^{-1}$. The magnitude of the numbers here are just for illustration. The relative values of the real and imaginary parts are the real point. The search will usually converge on some value, but it may not be meaningful. The search might also miss meaningful solutions if the initial guess is too far off.

In considering the outputs, you need to judge whether the imaginary value is small enough to be effectively zero, but this is usually pretty obvious. Looking at the energy diagnostics can often tell you if an answer is silly. The nonzero imaginary frequency means, also, that there can be very small energy exchanges in the energy diagnostics when the flow is stable. Further, if the contoured solutions show a concentration of contours near-surface in the center of the grid (where the arbitrary forcing occurs), then the result is likely bogus. Again, you need to decide what is effectively zero.

The search can sometime miss a complex frequency, for example if searching all takes place only near the real axis, but the true result has a substantial imaginary part. If it is a good solution, the inverse resonance parameter in the output (rrr) should be several orders of magnitude lower than neighboring values. Use a smaller accuracy tolerance (e) or start the search again farther from the real axis.

For an accuracy estimate ("e" option in jLWavesFinch), I find that 0.0001 works out fairly well. This means a nominal accuracy of 0.01% of the absolute value of the initial frequency guess.

The bottom stress is assumed to be proportional to the bottom velocity. For the errors associated with this assumption in the presence of a mean flow, see Brink (1997).

If more than one frequency is computed, there is an option to save the "dispersion curve" information (wavenumbers and complex frequencies).

When the Ertel potential vorticity $q = -N^2 f^*$ changes sign relative to f, symmetric instability is possible (Hoskins,1974). So, the routine "jLWavesRhocal" tests to see if f^*/f is negative anywhere. If this happens, f^* is contoured in red on the same plot as mean density, and a message appears on your Julia screen. If f^*/f does not become negative, it is not contoured, and no message about it is given. Experience with the program suggests that modes calculated when f^* changes sign are of questionable merit. Structures are often complex and have short scales near the region of negative f^*. It seems probable that if, in nature, a flow is symmetrically unstable, it will quickly mix to a neutral state with zero or positive f^*/f. So, even a "good" modal solution for a symmetrically unstable case is of questionable value.

Given the presence of the near-inertial spurious mode, and given that the "open" boundary condition is only reasonable for subinertial frequencies, the model results should only be trusted for subinertial frequencies, $|\mathrm{Re}(\omega)| < |f|$. If, in computing a dispersion "curve", you start to obtain real frequencies greater than the inertial, the superinertial results should be discarded.

All frequencies are in radians per second, and all wavenumbers in radians per cm. Radians do not have units, so these units could be written as "1/sec" for example.

You can get spurious results if the domain is too large relative to the natural scale of the wave under consideration. For example, with a domain size $L = 70$ km, trying to find a flat-bottom internal Kelvin wave with internal Rossby radius = 3 km yields bad results. Doing the same calculation with $L \leq 40$ km yields reasonable results. This problem arises because the arbitrary forcing used to create the resonance is applied near $L/2$. Trouble can arise when the desired wave has negligible amplitude at this distance offshore.

6. Disclaimer:

Although considerable effort has been made to make sure this software is correct and easy to use, there is no guarantee of perfection. If errors are found, or if documentation could be improved, please contact kbrink@whoi.edu .

References:

Brink, K.H., 1982. A comparison of long coastal trapped wave theory with observations off Peru. *Journal of Physical Oceanography*, **12**(8), 897–913.

Brink, K.H., 1989. The effect of stratification on seamount-trapped waves. *Deep-Sea Res*., 36, 825-844.

Brink, K. H., 1997. Time-dependent motions and the nonlinear bottom Ekman layer. *Journal of Marine Research*, **55**(4), 613–631.

Brink, K.H., 1999: Island-trapped waves, with application to observations off Bermuda. *Dyn. Atmos. Oceans*, 29, 93-118.

Brink, K. H., 2006. Coastal-trapped waves with finite bottom friction. *Dynamics of Atmospheres and Oceans*, **41**, 172-190.

Brink, K.H., 2023: *Physical Oceanography of the Continental Shelf*. Princeton University Press, 248 pp.

Dale, A.C., J.M. Huthnance, and T.J. Sherwin, 2001: Coastal-Trapped Waves and Tides at Near-Inertial Frequencies. , **31**, 2958–2970.

Hoskins, B.J., 1974: The role of potential vorticity in symmetric stability and instability. Quarterly Journal of the Royal Meteorological Society, 100(425), 480-482.

Mitchum G. and A.J. Clarke, 1986: The frictional nearshore response to forcing by synoptic scale winds. *J. Phys. Oceanogr.* 16: 934-946.

Mooers, C.N.K., 1975a: Several effects of a baroclinic current on the cross-stream propagation of inertial-internal waves. *Geophys. Fluid Dyn.,* 6, 245-275.

Mooers, C.N.K., 1975b: Several effects of a baroclinic current on the three-dimensional propagation of inertial-internal waves. *Geophys. Fluid Dyn*., 6, 277-284.

Wang, D.-P. and C.N.K. Mooers, 1976: Coastal-trapped waves in a continuously stratified ocean. *J. Phys. Oceanogr.,* 6: 853-863.